Chuleta de Examen — UD1: Arquitectura de Aplicaciones Web

Implantación de Aplicaciones Web (IAW)

---

1. Front-End vs Back-End: la base de todo

Toda aplicación web se divide en dos partes que se comunican entre sí. El **front-end** (cliente) es la capa que interactúa directamente con el usuario a través del navegador, construida con **HTML** (estructura), **CSS** (estilo) y **JavaScript** (comportamiento/interactividad). El **back-end** (servidor) contiene la lógica real: procesa datos, gestiona la base de datos y autentica usuarios, usando lenguajes como **Python, Java, Ruby o PHP** junto a sistemas gestores de bases de datos (**MySQL, PostgreSQL, MongoDB**). Esta separación es la razón por la que existen el resto de conceptos de la unidad (arquitectura cliente-servidor, tres niveles, protocolos): alguien tiene que **conectar** ambas partes, y eso se hace mediante **APIs** (Interfaz de Programación de Aplicaciones — conjunto de reglas que permite que dos programas distintos "hablen" entre sí) y los protocolos **HTTP/HTTPS**.

Patrones de diseño de la arquitectura web

Patrón	Qué resuelve	Idea clave
MVC (Modelo-Vista-Controlador)	Mezclar datos, interfaz y lógica en el mismo sitio	Separa **Modelo** (datos), **Vista** (interfaz) y **Controlador** (lógica/operaciones) → código más organizado y mantenible
SOA (Arq. Orientada a Servicios)	Reutilización y acoplamiento excesivo	Integra servicios **distribuidos, independientes y reutilizables**
Microservicios	Apps monolíticas difíciles de escalar/mantener	Descompone la app en servicios **independientes**, cada uno en su propio proceso, comunicados por protocolos como HTTP. Contrario al **monolito** (app como unidad indivisible): mejora escalabilidad y permite actualizar una parte sin tocar el resto

Buenas prácticas (muy preguntadas en examen)

- **Seguridad**: autenticación de usuarios, encriptación de datos, protección frente a **XSS** (Cross-Site Scripting, inyección de scripts maliciosos) y **CSRF** (Cross-Site Request Forgery, ejecución de acciones no autorizadas en nombre del usuario).
- **Optimización del rendimiento**: compresión de datos, minificación de CSS/JS, uso de **CDN** (redes de entrega de contenido) para acercar los recursos al usuario.
- **Responsive Design**: la interfaz se adapta a cualquier dispositivo/tamaño de pantalla.
- **Accesibilidad**: uso por personas con discapacidad — no solo buena práctica, es **requisito legal** en muchas jurisdicciones.

2. Aplicaciones Web vs. Aplicaciones de Escritorio

La causa de todas estas diferencias es una sola: dónde vive la aplicación. La app de escritorio se **instala** en un dispositivo concreto y explota directamente su hardware (CPU, RAM); la app web vive en un **servidor remoto** y se consume vía navegador, dependiendo por tanto de la conexión a internet.

Criterio	Aplicación Web	Aplicación de Escritorio
Acceso	Sin instalación, desde cualquier dispositivo con internet	Requiere descarga/instalación en un equipo concreto
Rendimiento	Depende de la conexión; mejorado gracias a HTML5/JS	Más rápida y eficiente: aprovecha al máximo CPU/RAM del equipo

Criterio	Aplicación Web	Aplicación de Escritorio
Seguridad / Actualizaciones	Actualización **automática** en el servidor, siempre última versión	Actualización **manual** por el usuario → riesgo de retraso en parches
Personalización	Más limitada, pero más fácil de iterar/mejorar sin intervención del usuario	Mayor personalización y funcionalidad a medida
Coste / Mantenimiento	Más económica a largo plazo (sin mantenimiento de hardware)	Coste inicial más alto + mantenimiento continuo de software y hardware

3. Arquitectura Cliente-Servidor

Es el modelo base de cualquier red: dos roles, el **cliente** (dispositivo o programa que **solicita** un servicio/recurso) y el **servidor** (programa/dispositivo que **provee** ese recurso). El funcionamiento es un ciclo de **petición** → **procesamiento** → **respuesta**, regulado siempre por un **protocolo de comunicación** común que garantiza que ambos extremos interpretan los datos igual.

Tipos de servidor según su función

- **Servidor Web**: entrega páginas web a los clientes (navegadores).
- **Servidor de Aplicaciones**: ofrece la lógica de negocio/servicios a las apps cliente.
- **Servidor de Base de Datos**: permite acceso y manipulación de bases de datos.

Ventajas frente a desafíos (relación causa-efecto)

Centralizar el sistema en un servidor trae **escalabilidad** (se pueden atender más clientes sin perder rendimiento), **mantenimiento centralizado** (actualizar un único punto simplifica la gestión) y **seguridad mejorada** (concentrar los datos permite reforzar mejor su protección). Pero esa misma centralización es también el origen de los desafíos: un **pico de solicitudes puede sobrecargar el servidor**, el sistema **depende de una red estable**, y al concentrar los datos en un único punto, **un ataque al servidor compromete todo el sistema** (mayor superficie de riesgo).

> ***Serverless**: modelo en la nube donde el proveedor gestiona toda la infraestructura de servidores; el desarrollador solo escribe código de aplicación y el proveedor se encarga de ejecutarlo, escalarlo y mantenerlo — reduce coste operativo y complejidad de gestión.*

La evolución de este modelo hacia el **IoT**, la **computación en la nube** y la integración de **IA/aprendizaje automático** amplía constantemente sus capacidades.

4. Arquitectura de Tres Niveles (3-tier)

Es un refinamiento de la arquitectura cliente-servidor: en vez de dos bloques (cliente/servidor), **separa explícitamente la lógica de negocio de los datos**, dando lugar a tres capas independientes que se pueden desarrollar, escalar y mantener por separado.

1. **Capa de Presentación**: interfaz con la que interactúa el usuario; su objetivo es mostrar datos y funcionalidades de forma clara y accesible.
2. **Capa de Lógica de Negocio**: el "núcleo" del sistema — procesa datos y ejecuta las reglas/procesos de negocio, garantizando que las operaciones se ejecuten correctamente.
3. **Capa de Datos**: gestiona almacenamiento, recuperación y manipulación de los datos, asegurando su integridad, seguridad y persistencia.

Separar responsabilidades en capas independientes genera directamente sus cuatro ventajas: **escalabilidad y flexibilidad** (modificar una capa sin tocar las demás), **mantenimiento y actualización simplificados**, **mejora del rendimiento** (la carga se reparte entre capas) y **seguridad reforzada** (aislar la capa de datos reduce accesos no autorizados).

Para implementarla correctamente hacen falta tres cosas: un **análisis y diseño detallado** (estudio de requisitos), una **selección de tecnologías adecuadas** para cada capa, y una **integración y pruebas rigurosas** que garanticen fiabilidad y estabilidad.

5. Protocolos de Aplicación: HTTP, HTTPS y FTP

Estos protocolos son las "reglas del idioma" que usan cliente y servidor para entenderse durante el ciclo de petición-respuesta visto en el punto 3.

Protocolo	Función	Seguridad	Puerto	Uso típico
HTTP (Hyper Text Transfer Protocol)	Base de la transferencia de datos en la WWW; rige cómo navegador y servidor intercambian mensajes	Sin cifrar	80	Navegación web estándar
HTTPS (HTTP Secure)	Variante segura de HTTP	Cifra con **SSL/TLS**	443	Transacciones sensibles: compras, banca electrónica
FTP (File Transfer Protocol)	Transferencia eficiente de **archivos** (no páginas web) entre sistemas TCP/IP	Depende de la variante (FTP simple no cifra)	21	Gestión de archivos en servidores: subir contenido, backups, archivos grandes

Por qué existe HTTPS: HTTP transmite los datos en texto plano, por lo que cualquiera interceptando la red podría leerlos. HTTPS añade una capa de cifrado (SSL/TLS) que protege la confidencialidad e integridad de la información — imprescindible cuando se manejan datos sensibles (contraseñas, tarjetas de pago).

Impacto en SEO: Google favorece y posiciona mejor los sitios que usan HTTPS por considerarlos más seguros para el usuario → mejora visibilidad y clasificación en buscadores.

FTP en la gestión web: herramienta clave para desarrolladores/administradores de sistemas — permite actualizar contenido, hacer copias de seguridad y mover archivos grandes de forma eficiente, tareas esenciales para mantener la integridad y el rendimiento de un sitio.

Métodos HTTP más comunes (contexto ampliado, útil para examen)

```
GET → solicita/recupera un recurso (p. ej. cargar una página)
POST → envía datos al servidor (p. ej. enviar un formulario)
PUT → actualiza un recurso existente
DELETE → elimina un recurso
```

6. Glosario técnico esencial

Término	Definición rápida
API	Conjunto de rutinas/protocolos que permite la interacción entre distinto software
Middleware	Software puente entre diferentes aplicaciones, o entre la aplicación y el hardware
Framework	Conjunto de herramientas/componentes estructurales para desarrollar software
Frontend	Parte de la app que interactúa con el usuario (interfaz y experiencia)
Backend	Parte de la app que corre en el servidor (datos y operaciones)
Servidor Web	Software/hardware que usa HTTP (y otros protocolos) para responder a peticiones de clientes web
Cliente Web	Programa (p. ej. navegador) que accede a servicios/recursos de un servidor web
Hosting	Servicio que permite publicar una app o sitio web en Internet
Dominio	Nombre único que identifica un sitio web en Internet

Término	Definición rápida
SSL (Secure Socket Layer)	Tecnología estándar para establecer un enlace cifrado entre servidor y navegador
Código fuente	Instrucciones escritas en un lenguaje de programación, legibles/editables por humanos
Base de datos	Colección organizada de datos, almacenada y accesible electrónicamente

7. Mapa mental de repaso rápido (orden de examen)

1. **App web vs escritorio** → dónde vive la app y quién la mantiene actualizada.
2. **Cliente-servidor** → modelo base: petición/respuesta + protocolo común.
3. **Tres niveles** → evolución del cliente-servidor separando Presentación / Lógica de negocio / Datos.
4. **HTTP / HTTPS / FTP** → el "idioma" que hace posible la comunicación entre las capas anteriores.
5. **Patrones (MVC, SOA, Microservicios)** → formas de organizar el código dentro de esas capas.
6. **Seguridad transversal**: XSS, CSRF, SSL/TLS, encriptación — aparece en todos los bloques anteriores.